Click-to-Source 3D

Stage 5 Report — Publishing Readiness and Automatic Tagging

From Locally-Buildable to Installable, and from Manual to Automatic

---

1. Executive Summary

Stage 5 was scoped as "package polish." It became the largest stage in the project, and it absorbed the one piece of work the roadmap had deliberately scheduled for after launch.

Two distinct bodies of work landed:

Publishing readiness. The repo went from "builds locally" to "installable by a stranger" — a ship-blocker fixed and then made impossible to reintroduce, four packages given real manifests, the source-edit write path extracted into its own package because it previously shipped in none, two security holes closed on an endpoint that could write any file under the project root, and a Three.js duplication that had been silently present since Stage 4.

Automatic tagging. Location stamping and per-instance capture both shipped, which the roadmap listed as Stage 6.5 and marked "optional, post-launch." The dogfooding application now runs entirely on automatic provenance for instanced objects — the hand-maintained arrays were removed, not merely shadowed by a precedence rule.

The headline result is narrow and verifiable: `Trees.jsx` and `GroundCover.jsx` lost 70 lines of provenance plumbing, and clicking any instance still resolves file, function, line, and per-instance transform arguments correctly. Parity against the removed arrays was 400/400 across all three tree meshes, all five fields, worst delta zero.

Test count went from 38 at the start of Stage 5 to 109 at its close.

---

2. Publishing Readiness

2.1 The ship-blocker

Barrel files used extensionless re-exports — `export * from "./resolver"`. Node's ESM resolver rejects these, so a direct import of any package's built entry failed with `ERR_MODULE_NOT_FOUND`. It worked only because Vite tolerates the omission, meaning the packages would have broken on a consumer's first direct import.

Ten specifiers across four files. TypeScript does not rewrite module specifiers on emit, so the source must carry the extension the output needs; `moduleResolution: "bundler"` accepts explicit `.js` pointing at a `.ts` file, so no config change was required at the time.

The `.d.ts` output carries extensions too, which independently fixed type resolution for downstream consumers on `node16` or `nodenext`.

2.2 Enforcement, not just repair

Nothing prevented a new extensionless import being added. Two candidates were scoped: switch `moduleResolution` to `nodenext`, or add a lint rule.

The lint route proved genuinely unavailable rather than merely expensive — `typescript-eslint` hard-refuses TypeScript 7, and the workaround was a side-by-side TypeScript 6 install purely for linting, at 75 packages and 47 MB.

`nodenext` was verified clean across all four packages, with `dist` byte-identical across all 26 emitted files by sha256. The negative control — reintroducing one extensionless specifier — produced `TS2835` on an ordinary `tsc -b`, not a flag-override probe. Enforcement that requires remembering a special command is not enforcement.

A side effect worth noting: the `paths` block became inert and was removed, so cross-package resolution now goes through `node_modules` and the exports maps — the same route a real consumer takes. That is precisely the class of gap that had produced an `ERR_PACKAGE_PATH_NOT_EXPORTED` regression earlier in the stage.

2.3 Publishing manifests

All packages were `"private": true` at version `0.0.0`, with only `main` and `types` — no `exports`, no `files`, no prepublish step.

Three defects would have shipped:

`publishConfig: { access: "public" }` **was missing.** Scoped packages default to restricted, so publish would have failed outright.

`core` **declared** `three` **in** `dependencies` **rather than** `peerDependencies`**.** Because `^0.170.0` on a 0.x version resolves to `>=0.170.0 <0.171.0`, it cannot satisfy a consumer on 0.173, so npm nests a second copy. The evidence it was already happening was a hand-written `"overrides": { "three": "$three" }` in the consuming app — a workaround whose existence was the symptom.

The tarball was correct only by accident. `dist/` is gitignored, so a publish from a clean checkout would have shipped `src/` plus a `tsconfig.json` whose `extends` pointed outside the tarball, with `main` aimed at a file that was not there. The `prepack`-versus-`prepublishOnly` distinction mattered: putting the build in `prepublishOnly` means `npm pack --dry-run` inspects stale output, so the verification step lies.

Peer ranges were also corrected. The declared `@react-three/fiber >=8.0.0` crossed with `react >=18.0.0` permits pairings that cannot work — R3F v8 needs React 18, v9 needs React 19 — and only fiber 9 had ever been tested.

2.4 The vite-plugin extraction

The source-edit write path — the `/__cts/read-file` and `/__cts/write-file` endpoints — shipped in no published package. The consuming app carried a hand-copied `vite-file-io.js`, which was itself the evidence that consumers needed it and had no supported way to get it.

An npm consumer would have installed a panel that resolves provenance and cannot write anything back. That is the difference between a viewer and the tool as described.

The plugin became `@click-to-source/vite-plugin`, and the Babel-based source editor moved with it. That second decision was not tidiness: `editSource` had exactly one non-test consumer and it runs in Node. Babel was in the browser bundle purely as barrel collateral.

The result made `core` browser-pure — its entire production closure is now `@click-to-source/shared` alone — and removed `@babel/parser` (2.0 MB) and `magic-string` (480 KB) from every consumer's bundle.

Timing mattered. This was a breaking change to `core`'s public surface, free at 0.1.0-unpublished and awkward forever after.

2.5 Security

Two defaults were tolerable when the plugin was a hand-installed per-project file and were not once a one-line npm install broadened the exposure.

No writable-file allowlist — any file under the project root could be overwritten, including `package.json` and `.env`. The allowlist was derived from Babel's actual parse capability rather than arbitrary policy.

A test of `path.extname` behaviour changed the design. `.env` was rejected, but by accident: Node returns `""` for a dot-leading basename rather than `".env"`, so it fails the allowlist rather than matching a deny rule. The genuine gap was that `.babelrc.js` and `.hidden.ts` *passed*. A dot-prefixed-basename rejection was added alongside the extension check, with the `.env` case given an explicit regression test so nobody "fixes" the accidental behaviour later.

No origin check — any page in the developer's browser could POST to `localhost:5173/__cts/write-file`. The negative control that demonstrated the fix also demonstrated the hole was real: it wrote an actual `.env` containing `STOLEN=1` before the check was in place.

2.6 Three.js duplication

The consuming app was bundling two copies of Three.js. Baseline `__THREE__` marker counting gave 2 for a known-single-copy app and 4 for this one, cross-checked against both revision literals.

The `overrides` workaround had never addressed it. npm's overrides govern npm's tree, which the symlink path respects — but Vite defaults to `preserveSymlinks: false`, resolves to the real path inside the monorepo, and walks up into the workspace's own devDependency. The override never entered that code path.

This predated the session. Every `SelectionHighlight` post-processing pass during Stage 4 and the H4/H6 experiments ran on 0.170 against objects the app created with 0.173. It worked, which is why it went unnoticed for so long.

`resolve.dedupe: ['three']` fixed it: `__THREE__` 4 to 2, bundle 1,447.09 to 1,327.69 kB raw and 408.22 to 374.92 kB gzip, and the "Multiple instances of Three.js" warning stopped.

A tarball test settled whether the override was still needed for published consumers: installing all four packages into a scratch consumer outside both repos produced a single `three`, correct from every vantage point including realpath. The override was removed — though not for the reason first assumed. It was inert against the click-to-source duplication, but it *was* collapsing a genuine second copy pulled by `stats-gl@2.4.2`, which declares `three: "*"`.

2.7 Toolchain hygiene

Binary attributes. All 12 tracked PDFs were subject to CRLF corruption — `core.autocrlf=true`, no `.gitattributes`, and no NUL bytes in the first 8000 so git classified them as text. PDF xref tables store absolute byte offsets, which break when the checkout filter adds a byte per line. The verification caught a live instance: one report on disk carried 277 carriage returns and its `startxref` pointed into the middle of stream data. The committed blob was correct, so only the local checkout was damaged.

Prepack pruning. `tsc -b --force` rebuilds but never prunes. After the editor moved out of `core`, a forced rebuild left `dist/sourceEditor.js` and its `.d.ts` in place — and with `files: ["dist"]`, those orphans would have shipped in a package that had just declared those dependencies removed. Demonstrating the fix required `--ignore-scripts` to avoid the circularity of testing a prepack script with a command that runs it.

Vitest. All five audit findings traced to `vitest@2.1.9`. The critical (CVSS 9.8) required `vitest --ui`, which is not installed; the esbuild moderate required `--serve` mode, which Vite does not use. Neither was reachable. The upgrade to 4.1.11 needed no code changes, cleared all five, dropped `vite-node` and `esbuild` entirely, and aligned a split where tests imported `createServer` from top-level Vite 8 while vitest ran internally on a bundled Vite 5.

A `pretest` bug found along the way. Three test files import `@click-to-source/shared` at runtime, but the test scripts had no build dependency — so a fresh clone failed 3 files and 30 tests. Found only because the near-miss on the upgrade forced a check of whether the current setup had the same fragility.

---

3. Mesh Attributes

The overlay store already held `selectedObject` and `instanceId`; `GenerationTrace` read neither, rendering only `sourceRef`.

Everything added is a read off a live object at click time — no capture, no build tooling, and structurally unable to drift, which directly addresses the failure mode behind three Stage 4 bugs.

What the panel now shows: instance index and count, object type and uuid, `renderOrder`, `frustumCulled`, vertex and triangle counts, indexed status, attributes present, bounds, material type and colour, and the transparency flags.

Three findings changed the design, all from running things rather than reasoning:

`InstancedMesh.type === "Mesh"`**.** `object.type` cannot distinguish instanced from regular; the discriminator is `isInstancedMesh`. Without this the panel would have confidently reported "Mesh" for a tree.

Bounding volumes are `null` by default, and the panel must not compute them. Calling `computeBoundingBox()` from an inspector mutates the object being inspected, destroying the property that makes the feature worth having. The panel displays "not computed" instead, and a test asserts the absence of that side effect.

The instance matrix is more accurate than the stored args. Float32 readback carries roughly 1e-6 error, but the hand-authored args stored `.toFixed(3)` — and that rounding is larger. The stored value was the lossy one.

A follow-on fix was needed: the panel outgrew its container, truncating mid-content with no way to reach the rest. A measurement harness caught a subtlety on its first run — `max-height` applies to the content box, so a 680px cap rendered at 712px under default `content-box` sizing.

---

4. Automatic Tagging

4.1 Location stamping

The decisive finding was that the data already existed. `@vitejs/plugin-react` injects `__source` into every JSX element in dev, and R3F throws it away: `applyProps` iterates the props, finds `__source` does not exist on `THREE.Mesh`, and assigns nothing.

So the transform rewrites `__source` into `userData.__ctsSource` at build time rather than generating anything new. Build-time rather than reading React's fiber at runtime, because React 19 removed `_debugSource` and broke that approach for several existing tools.

Precedence: manual beats auto, anywhere in the chain. This is not a hedge against auto being unreliable. All four tagged objects construct geometry inside `useMemo`, so the stamp lands on the `<mesh>` JSX line while the hand-written ref points at the declaration `editSource` can actually rewrite. If the nearer stamp won, editing `noiseFloor` would fail with `ARGUMENT_NOT_FOUND`.

Two defects were found and fixed during implementation:

DOM elements were being stamped, and React errored on every one — `App.jsx` carried 15 spurious `userData` attributes. The fix uses a denylist rather than an allowlist, because `extend()` makes the R3F element set open-ended and the dogfooding app's own `<waterMaterial>` is a live example an allowlist would have missed.

Paths were absolute. `__source` carries filesystem paths, so the panel would display the developer's directory structure on any screenshot or shared session. Normalization was applied unconditionally, not just in production builds.

4.2 Per-instance capture

The H4/H6 composition, shipped essentially as the experiments proved it.

Routing patches `InstancedMesh.prototype.setMatrixAt`. The receiver is the destination mesh and the first argument is the slot — both are the call's own data, nothing derived. Constructor identity-fill is filtered out, since Three's constructor calls `setMatrixAt(i, _identity)` for every slot and a prototype patch therefore sees double traffic.

Arguments come from a chain: `updateMatrix` capture, then `Matrix4.prototype.clone()` interception, joined at `setMatrixAt` on the clone's object identity. The clone is where distinct identity comes into existence — `dummy.matrix` is the same object every iteration.

The pairing filters by receiver identity, not by count. The clone ratio is 2:1 in Trees and 1:1 in GroundCover, so any count-based rule is codebase-specific and would ship a silent failure. This was tested by injecting one foreign `Matrix4.clone()` per instance: the counting rule broke outright while receiver-filtered pairing held.

The probe installs as a side effect on import, because the timing constraint makes an explicit call unusable — a consumer calling it in a component body is already too late. Writes are once-only with no replay.

Precedence is per-instance, not per-mesh, so a partially-populated manual array leaves uncovered slots resolving via capture rather than resolving to nothing.

4.3 Verification

The dogfooding app could not demonstrate either feature, because every object in it was hand-tagged and therefore always took the manual branch. Both required temporarily removing the manual path to see the automatic one work.

For location stamping, an untagged mesh was added: it resolved to `src/components/Scene.jsx · Scene · line 108`, project-relative, with no `sourceRef` anywhere.

For instance capture, the arrays were stripped and a real raycast performed — instance 149 was the first intersection, a genuine top-level click, and resolved through the registry with correct distinct args.

The decisive check was parity. With the manual arrays still present, captured values were compared against them across every slot: **400/400 across all three tree meshes, all five fields, worst delta 0, no slot missing a record.** The probe independently reconstructs exactly what the developer had maintained by hand.

This is a standing limitation, not a one-off inconvenience: the app cannot regression-test either path while its objects remain tagged. Coverage lives in unit tests instead.

4.4 Removing the manual arrays

The arrays were removed from `Trees.jsx` and `GroundCover.jsx` — 70 lines deleted, 13 added, net –57. This is the change that proves the feature carries the app rather than sitting behind a precedence rule that always favours the manual path.

One field was lost: `variant`**.** `const variantRoll = rng()` selects which array — hence which mesh — a matrix goes into. It never enters the matrix, so no transform-based capture can route it. Grass-versus-bush separation survives via distinct stamped lines; variant-within-kind does not.

This is the worked example of a structural limit rather than an implementation gap, and it matches what H2 measured: values that cross no call boundary and reach no matrix are invisible to every mechanism tested.

4.5 Cost

Benchmarked against the shipped implementation rather than cited from the experiments:

	unpatched	patched	factor
`setMatrixAt`	~55 ns	560–676 ns	10–12x
`Matrix4.clone`	171–220 ns	3,277–3,587 ns	**15–21x**

Roughly 4.4–4.7 ms for a 1,205-instance scene, against the recorded experiment's 2.6 ms.

The clone multiplier dominates, and the cause is a design decision: `clone()` decomposes eagerly into position, quaternion, scale and Euler on every `Matrix4.clone()` in the process, even though most clones never reach a `setMatrixAt`. Storing the sixteen elements and decomposing lazily on read would move that off the hot path.

It has not bitten yet only because `clone()` is quiet in the codebases measured — which is a property of those codebases, not a guarantee. Both patches are process-global and affect every `InstancedMesh` in the process, including library-owned ones.

---

5. What Auto-Tagging Covers

Works, verified in a browser, zero annotation:

- File, function and line on any JSX element, tagged or not
- Which mesh and which instance index, for hand-rolled `InstancedMesh`
- Per-instance transform arguments — position, scale, rotation
- Mesh state: geometry counts, attributes, material properties, render flags

Does not cover:

- Values that never reach the transform — `variant`, and by the same reasoning `suitability`, `altNorm`, `altitudeFactor`
- `seed` and generator constants, which are capturable but need a hook at generator entry that has not been built
- Anything inside GLSL

Scope: React Three Fiber only. Location stamping is R3F-only by construction, since it rewrites JSX `__source` and raw Three.js has no JSX to stamp.

A note on drei. An investigation concluded that `<Instances>` needs nothing built. It has no programmatic API — `instances` is `useState` populated only through `subscribe`, called by `Instance` on itself. `PositionMesh.raycast` sets `intersect.object` to the individual instance, so a `userData` prop resolves through the existing parent-walk. This narrows, rather than supersedes, the earlier conclusion that library-owned instancing was unreachable: the H4/H6 composition genuinely cannot reach it, but it does not need to.

---

6. What Is Left

6.1 Immediate follow-ups

Item	Note
Lazy decomposition in the clone patch	The measured 15–21x is the strongest argument for it
Live-versus-args drift comparison	Deferred from the mesh panel; `describeMesh` is the seam
Shared geometry/material detection	Deferred; needs a `scene.traverse` per selection

Unvalidated editable args	`TODO(follow-up)` in `GenerationTrace` — an unresolvable arg renders as editable and only fails on save
`describeMesh` on drei instances	Reports kind `Group`, `triangleCount: null`, `materials: []`, since `PositionMesh extends Group`. Cosmetic.
`variant`, if wanted back	The fix is `userData.sourceRef.args` on the mesh, not a better probe

6.2 Remaining stages

Stage 6 — MCP server. Exposes the resolution engine as agent-callable tools. The capture registry is now exactly the queryable index `search_by_generator` needed and nothing previously built. `search_by_seed` still requires seed capture, which is unbuilt — so that tool either ships without it or seed capture gets pulled forward.

Stage 7 — ship. npm publish, MIT, narrow launch. The packages are verified installable from tarballs, so the mechanical work is done. Publishing retires the consuming app's `file:` links and its `resolve.dedupe` workaround. It also makes the `exports` maps a contract and consumes version numbers — breaking changes that are free now stop being free.

Three.js support. Three pieces. Instance capture likely works already, since it patches plain Three.js prototypes with nothing React-specific. Location needs a different transform matching `NewExpression` against Three.js constructors — closer to the call-site rewriting tested as H2, which measured 78% false positives on a naive directory glob. And the overlay needs a non-React path, since `GenerationTrace` is a React component and `useClickToSource` is a hook. The resolver in `core` is already framework-agnostic, so the engine half is portable.

This is a stage, not a task, and there is no raw Three.js dogfooding target to verify against.

6.3 Future capabilities

Capability	Requires
Seed and generator constants	Capture at generator entry; no routing needed since it does not vary per instance
Placement predicates	H2-style call capture, clock-joined to the clone chain. Untested but with a clear path — H2 captured these at 200/200 and could not route them; the chain now supplies a key.
Value provenance and blast radius	Static AST resolve pass; generalises the `terrainSize` incident
Shader uniforms set from JS	Tracing JS-side assignment — the tractable half of shader support
Values inlined in GLSL	A GLSL parser and a patcher that round-trips shader source safely. Its own project.

---

7. Assessment

Stage 5 was scoped as polish and turned out to be the difference between a project that builds and a project that ships. Nearly everything in Section 2 was invisible from inside the repo — the packages built, the tests passed, the app ran, and none of that would have survived a stranger typing `npm install`.

The recurring pattern is that verification kept finding what reasoning had missed. The tarball was correct only because `dist` happened to exist locally. The override was collapsing a duplicate, but not the one it was blamed for. `.env` was rejected, but by accident, while `.babelrc.js` passed. The count-based clone rule worked on this codebase and broke the moment a foreign clone was injected. In each case the check that caught it existed because the claim was tested rather than argued.

The auto-tagging result is the one worth stating plainly. Six experiments established that capture and routing are independent axes and that only destination-side recording can route. That research became shipped code, and the dogfooding application now runs on it — 70 lines of hand-maintained provenance deleted, parity verified at 400/400 with zero delta, one field lost for a documented structural reason.

The original goal was to click a mesh and get the code without writing provenance by hand. For React Three Fiber, that now happens.